

VSS

Vectorized Signal Star

A SIMD-lite extension of the StarCore-1 RISC core

EEE4120F HPES Project · 2026

Group 7

Joab Gray Kloppers KLPJOA002

Alex Hillman HLLALE010

Justin Heathcote-Marks HTHJUS001

Twice the byte throughput. One instruction.

VSS adds a single SIMD-lite extension to the Practical 4 StarCore-1: a new VECTOR opcode that splits the 16-bit ALU into two independent 8-bit lanes. Built for the byte-packed workloads embedded telemetry lives on — sensor fusion, status masks, threshold compares — where the scalar core wastes most of its cycles unpacking and repacking.

The problem with scalar on packed data

Why specialise the StarCore-1 at all?

StarCore-1 (baseline)

Two bytes packed in one register? You pay 5–8 instructions per pair.

```
; add packed pairs A, B
AND  R1, A, 0xFF00    #Mask upper 8 Bits For A
AND  R2, A, 0x00FF    #Mask Lower 8 Bits For A
AND  R3, B, 0xFF00    #Mask upper 8 Bits For B
AND  R4, B, 0x00FF    #Obtain upper 8 Bits For B
ADD  R5, R1, R3        #Add upper 8 Bits
ADD  R6, R2, R4        #Add lower 8 Bits
OR   R7, R5, R6        #Mask The Two Together
```

7 cycles per byte-pair

VSS (this work)

One instruction, both bytes, one cycle.

```
; add packed pairs A, B
```

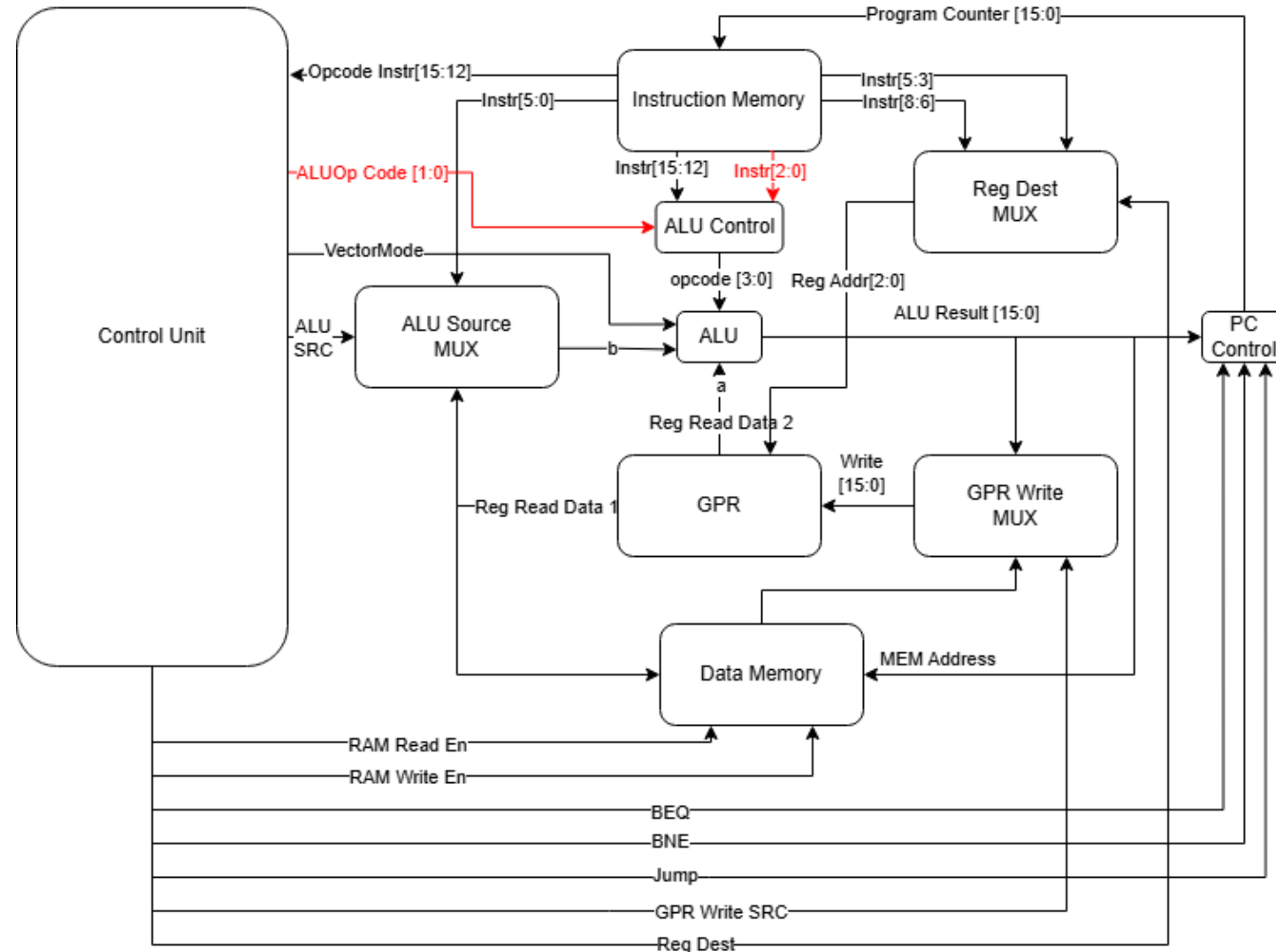
```
VADD R7, A, B
```

1 cycle per byte-pair

Telemetry payloads pack two ADC channels or status bytes per 16-bit word. *The scalar core spends most of its life moving bits around — VSS does the actual work instead.*

The StarCore-1 datapath with VSS modifications

Red signals are new for VSS; everything else is unchanged from Practical 4.



What's new

vector_mode

New 1-bit signal from Control Unit. Switches ALU between scalar and 2×8 -bit lane mode.

ALUOp = 2'b11

New encoding (ALUOP_VECTOR). Tells ALU_Control to bypass the opcode decode table.

funct[2:0]

`instr[2:0]` of the VECTOR instruction; passes straight through to `alu_control`.

opcode 4'b1110

VECTOR instruction. R-type-like register addressing, writes ALU result back to register file.

Three modules

The smallest possible edit that delivers a real SIMD extension.

ALU.v

Lane-split arithmetic

Dual case-statement: `vector_mode = 0` keeps the original 16-bit ops; `vector_mode = 1` splits `a`, `b` and result into independent {hi, lo} 8-bit lanes. The carry chain between bits 7 and 8 is broken by construction — each lane evaluates as its own 8-bit expression.

-
- + `vector_mode` input
 - + 8 SIMD-lite ops

ALU_Control.v

Funct passthrough

Added one casex pattern: `6'b11xxxx` → `ALU_Cnt = funct`.
When `ALUOp = 2'b11` the `funct` field bypasses the opcode decode table entirely. Every baseline pattern from Practical 4 is unchanged.

-
- + `funct[2:0]` input
 - + `ALUOP_VECTOR` case

ControlUnit.v

New opcode + mode bit

Added case for opcode `4'b1110` (VECTOR): R-type-like register addressing, `ALUOp = 2'b11`, and assertion of the new `vector_mode` output. All other opcodes safely default `vector_mode` to 0.

-
- + `vector_mode` output
 - + `OP_VECTOR` case

The VECTOR instruction

One opcode, eight operations, zero impact on existing instructions.

Instruction encoding (16 bits)



funct[2:0] selects the vector operation

funct	Mnemonic	Behaviour (when vector_mode = 1)
000	VADD	lane-wise add
001	VSUB	lane-wise sub
010	VINV	bitwise NOT of operand A
011	VSHL	per-lane shift left (b[11:8] b[3:0])
100	VSHR	per-lane shift right (b[11:8] b[3:0])
101	VAND	bitwise AND
110	VOR	bitwise OR
111	VSLT	per-lane set-less-than (unsigned)

Reusing the ALU's existing 3-bit operation encoding for funct means ALU_Control needs no new decode logic — it just forwards the bits. Opcode 4'b1010 (the brief's reserved coprocessor slot) is deliberately left untouched.

Gold standard: software reference vs hardware DUT

Two independent implementations. If they agree, the hardware matches the spec.

Verification flow

1

Python golden model

vss_golden.py — independent reference implementation of the VSS ALU spec.

2

Test vector list

14 ALU + 16 ALU_Control + 9 ControlUnit cases, covering scalar regression and every vector op.

3

Verilog testbench

Drives the same vectors into the DUT, compares to expected values, prints PASS/FAIL.

4

VCD waveform

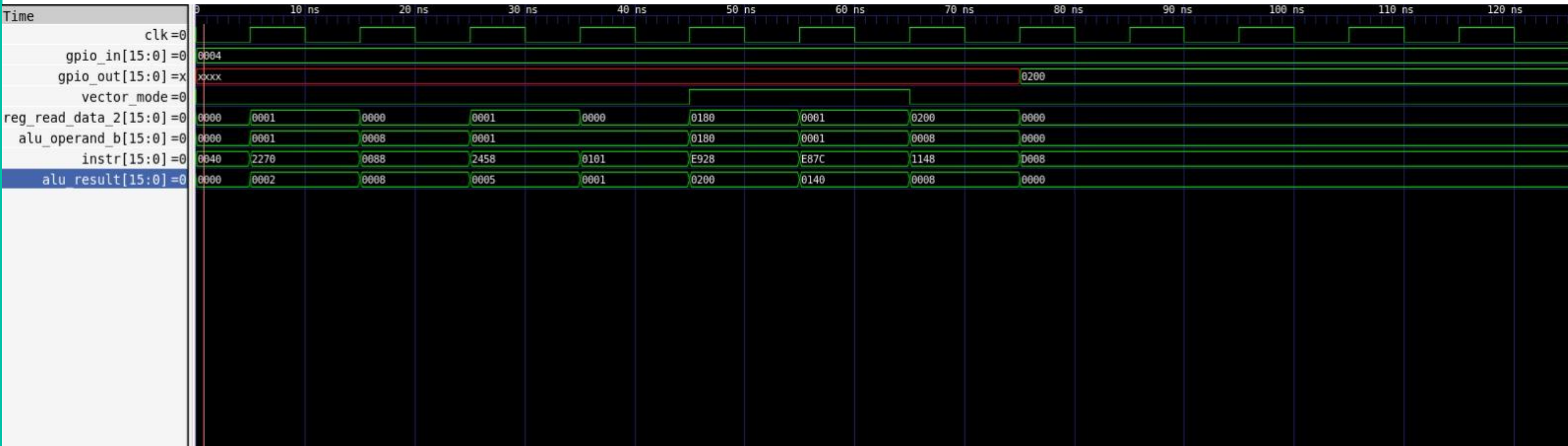
\$dumpvars captures every signal — GTKWave shows the lane split visually.

```
vss_golden.py · alu_vector(a, b, op)
```

```
def alu_vector(a, b, op):  
    a_hi, a_lo = (a >> 8) & 0xFF, a & 0xFF  
    b_hi, b_lo = (b >> 8) & 0xFF, b & 0xFF  
  
    if op == ADD:  
        hi = (a_hi + b_hi) & 0xFF # carry trapped  
        lo = (a_lo + b_lo) & 0xFF # wraps in-lane  
    elif op == SUB:  
        hi = (a_hi - b_hi) & 0xFF  
        lo = (a_lo - b_lo) & 0xFF  
    ...  
  
    return ((hi << 8) | lo) & 0xFFFF
```

Waveform proof:

Same operands, two cycles. The signal trace makes the lane split self-evident.



Live demo — what we'll run now

1

`YODA_tb.v`

Star-Core 1 with SIMD

1Adapting Star-Core 1 Normal Test bench

2

`gtkwave`

Waveform inspection

VSS, in one slide

What we built

A SIMD-lite extension of StarCore-1: one new opcode, one new mode signal, 8 vector operations sharing the existing 3-bit ALU encoding.

What we proved

39 of 39 module-level golden tests pass. Scalar regression is zero. The lane split is visible on the waveform between bits 7 and 8.

What's next

Top-level integration with the unchanged Practical 4 datapath, then memory-mapped I/O for telemetry ingest. Per-lane masks and saturation as stretch goals.

Thank you.

Questions?

Group 7 · EEE4120F HPES Project · 2026

Joab Gray Kloppers (KLPJOA002) · Alex Hillman (HLLALE010)